

PANDUAN PESERTA

Kontainerisasi Laravel hingga Produksi di Kubernetes

Aspek	Keterangan
Durasi	4 hari ($\pm$ 24 jam efektif)
Tingkat	Menengah
Jumlah latihan	21 latihan mandiri
Peran dokumen	Buku kerja peserta

Daftar Isi

Daftar Isi.....	2
Cara Memakai Buku Kerja Ini.....	6
Struktur Tiap Latihan.....	6
Daftar Latihan	6
Hari 1 — Fondasi: dari Aplikasi Menjadi Image	7
L1.1 Mengurutkan Ulang Dockerfile	8
Soal.....	8
Petunjuk.....	8
Konsep yang Dilatih.....	8
Ruang Jawaban	8
L1.2 Membaca Pembagian Hak Akses PHP-FPM.....	9
Soal.....	9
Petunjuk.....	9
Konsep yang Dilatih.....	9
Ruang Jawaban	9
L1.3 Membuktikan Perkakas Build Tidak Ikut Terbawa	10
Soal.....	10
Petunjuk.....	10
Konsep yang Dilatih.....	11
Ruang Jawaban	11
L1.4 Menemukan Berkas yang Seharusnya Tidak Ikut.....	12
Soal.....	12
Petunjuk.....	12
Konsep yang Dilatih.....	12
Ruang Jawaban	12
Refleksi Hari 1	13
Hari 2 — Stack Lengkap dengan Docker Compose	14
L2.1 Menambah Service Baru ke Stack	14
Soal.....	14
Petunjuk.....	14
Konsep yang Dilatih.....	15
Ruang Jawaban	15
L2.2 Membedakan Container "Up" dan Database "Siap"	16

Soal.....	16
Petunjuk.....	16
Konsep yang Dilatih.....	16
Ruang Jawaban	16
L2.3 Menentukan Kebijakan Eviction.....	17
Soal.....	17
Petunjuk.....	17
Konsep yang Dilatih.....	17
Ruang Jawaban	18
L2.4 Melacak Job yang Gagal	19
Soal.....	19
Petunjuk.....	19
Konsep yang Dilatih.....	19
Ruang Jawaban	19
L2.5 Cache Konfigurasi di Waktu yang Salah	20
Soal.....	20
Petunjuk.....	21
Konsep yang Dilatih.....	21
Ruang Jawaban	21
L2.6 Menguji Paritas Dua Web Server	22
Soal.....	22
Petunjuk.....	22
Konsep yang Dilatih.....	22
Ruang Jawaban	22
L2.7 Mengukur Dampak Penempatan vendor/	23
Soal.....	23
Petunjuk.....	23
Konsep yang Dilatih.....	24
Ruang Jawaban	24
Refleksi Hari 2	25
Hari 3 — Kubernetes: Konsep Inti.....	25
L3.1 Menelusuri Deployment sampai Endpoint.....	26
Soal.....	26
Petunjuk.....	26

Konsep yang Dilatih.....	26
Ruang Jawaban	26
L3.2 Perubahan Konfigurasi yang Tidak Berlaku.....	27
Soal.....	27
Petunjuk.....	27
Konsep yang Dilatih.....	27
Ruang Jawaban	27
L3.3 Menguji Ketahanan Data.....	29
Soal.....	29
Petunjuk.....	29
Konsep yang Dilatih.....	29
Ruang Jawaban	29
L3.4 Memilih Probe yang Tepat	30
Soal.....	30
Petunjuk.....	30
Konsep yang Dilatih.....	30
Ruang Jawaban	30
L3.5 Membuat Overlay Staging.....	31
Soal.....	31
Petunjuk.....	32
Konsep yang Dilatih.....	32
Ruang Jawaban	32
Refleksi Hari 3	33
Hari 4 — Produksi dan Keamanan	34
L4.1 Memperbaiki Manifest yang Ditolak.....	34
Soal.....	34
Petunjuk.....	34
Konsep yang Dilatih.....	34
Ruang Jawaban	34
L4.2 Menemukan Direktori yang Perlu Writable	35
Soal.....	35
Petunjuk.....	36
Konsep yang Dilatih.....	36
Ruang Jawaban	36

L4.3 Menguji Apakah NetworkPolicy Benar-benar Ditegakkan	37
Soal.....	37
Petunjuk.....	37
Konsep yang Dilatih.....	37
Ruang Jawaban	37
L4.4 Mengukur Kegagalan Request Selama Rollout	38
Soal.....	38
Petunjuk.....	38
Konsep yang Dilatih.....	39
Ruang Jawaban	39
L4.5 Analisis Akar Masalah dari Log	40
Soal.....	40
Petunjuk.....	40
Konsep yang Dilatih.....	40
Ruang Jawaban	40
Refleksi Hari 4	41

Cara Memakai Buku Kerja Ini

Buku kerja ini berisi latihan mandiri. Berbeda dengan modul yang dipegang fasilitator, di sini TIDAK ada perintah yang sudah jadi untuk disalin. Yang tersedia hanya soal, petunjuk minimal, dan ruang untuk menuliskan temuan Anda sendiri.

Itu disengaja. Menyalin perintah menghasilkan layar yang benar tanpa pemahaman yang ikut terbentuk. Menyusun sendiri perintahnya — lalu salah, lalu memperbaikinya — jauh lebih lambat dan jauh lebih bertahan lama.

Struktur Tiap Latihan

Bagian	Isi
Soal	Tugas atau pertanyaan yang harus diselesaikan
Petunjuk	Arah, bukan langkah. Cukup untuk memulai, tidak cukup untuk menyalin
Konsep yang Dilatih	Konsep dari modul yang sedang diuji latihan ini
Ruang Jawaban	Pertanyaan pengamatan dan analisis — isi dengan tulisan tangan

Sebagian pertanyaan menuntut Anda **MENYIMPULKAN**, bukan sekadar melaporkan apa yang terlihat di layar. Pertanyaan seperti itu tidak punya satu jawaban tunggal yang benar; yang dinilai adalah penalarannya.

CARA MENERJAKAN YANG DISARANKAN

Baca seluruh soal sampai selesai sebelum mengetik satu perintah pun.

Kerjakan dulu tanpa membuka modul. Buka modul hanya bila benar-benar buntu.

Tulis apa yang Anda **LIHAT**, bukan apa yang Anda harapkan terlihat. Selisih antara keduanya adalah bahan belajar yang paling berharga.

Bila hasilnya berbeda dengan dugaan, jangan langsung mengulang. Cari tahu dulu kenapa.

Daftar Latihan

Kode	Judul	Hari
L1.1	Mengurutkan Ulang Dockerfile	Hari 1
L1.2	Membaca Pembagian Hak Akses PHP-FPM	Hari 1
L1.3	Membuktikan Perangkat Build Tidak Ikut Terbawa	Hari 1

L1.4	Menemukan Berkas yang Seharusnya Tidak Ikut	Hari 1
L2.1	Menambah Service Baru ke Stack	Hari 2
L2.2	Membedakan Container "Up" dan Database "Siap"	Hari 2
L2.3	Menentukan Kebijakan Eviction	Hari 2
L2.4	Melacak Job yang Gagal	Hari 2
L2.5	Cache Konfigurasi di Waktu yang Salah	Hari 2
L2.6	Menguji Paritas Dua Web Server	Hari 2
L2.7	Mengukur Dampak Penempatan vendor/	Hari 2
L3.1	Menelusuri Deployment sampai Endpoint	Hari 3
L3.2	Perubahan Konfigurasi yang Tidak Berlaku	Hari 3
L3.3	Menguji Ketahanan Data	Hari 3
L3.4	Memilih Probe yang Tepat	Hari 3
L3.5	Membuat Overlay Staging	Hari 3
L4.1	Memperbaiki Manifest yang Ditolak	Hari 4
L4.2	Menemukan Direktori yang Perlu Writable	Hari 4
L4.3	Menguji Apakah NetworkPolicy Benar-benar Ditegakkan	Hari 4
L4.4	Mengukur Kegagalan Request Selama Rollout	Hari 4
L4.5	Analisis Akar Masalah dari Log	Hari 4

Hari 1 — Fondasi: dari Aplikasi Menjadi Image

LATIHAN HARI 1

L1.1 — Mengurutkan Ulang Dockerfile

L1.2 — Membaca Pembagian Hak Akses PHP-FPM

L1.3 — Membuktikan Perkakas Build Tidak Ikut Terbawa

L1.4 — Menemukan Berkas yang Seharusnya Tidak Ikut

L1.1 Mengurutkan Ulang Dockerfile

Soal

Sebuah tim mengeluh: setiap kali mereka mengubah satu baris kode PHP, build image memakan waktu lebih dari empat menit karena Composer mengunduh ulang seluruh dependensi.

Perbaiki Dockerfile di bawah agar perubahan kode tidak lagi membatalkan pemasangan dependensi, lalu BUKTIKAN perbaikan Anda dengan pengukuran.

```
FROM php:8.4-fpm-alpine
WORKDIR /var/www/html

COPY src/ ./
COPY --from=composer:2 /usr/bin/composer /usr/bin/composer
RUN composer install --no-dev --prefer-dist
```

Petunjuk

- Cache sebuah layer batal bila instruksinya berubah ATAU ada layer di bawahnya yang berubah.
- Ubah hanya URUTAN instruksi. Jangan menambah atau menghapus perintah.
- Ukur dengan membangun dua kali, lalu ubah satu berkas kode dan bangun lagi.

Konsep yang Dilatih

Layer image · Cache build · Urutan instruksi Dockerfile

Ruang Jawaban

L1.1.1 Tulis ulang urutan instruksi yang benar.

L1.1.2 Berapa detik build kedua SEBELUM perbaikan, dan berapa SESUDAH perbaikan?

L1.1.3 Instruksi mana yang paling mahal, dan kenapa posisinya menentukan?

L1.2 Membaca Pembagian Hak Akses PHP-FPM

Soal

Jalankan sebuah container php-fpm, lalu amati daftar proses di dalamnya.

Jelaskan mengapa proses-proses itu TIDAK berjalan sebagai user yang sama, dan apa akibatnya bagi keamanan.

Petunjuk

- Perintah untuk melihat proses beserta pemiliknya tersedia di dalam image Alpine.
- Perhatikan kolom user pada baris master dan baris pool.

Konsep yang Dilatih

Arsitektur PHP-FPM · Drop privilege · Prinsip hak akses minimal

Ruang Jawaban

L1.2.1 Salin dua baris pertama keluaran daftar proses.

L1.2.2 User apa yang menjalankan proses master, dan user apa yang menjalankan pekerja?

L1.2.3 Kenapa master perlu hak akses yang lebih tinggi daripada pekerja?

L1.2.4 Apa risikonya bila SEMUA proses dijalankan sebagai root?

L1.3 Membuktikan Perkakas Build Tidak Ikut Terbawa

Soal

Sebuah image produksi diklaim "sudah multi-stage dan bersih".

Buktikan secara mandiri apakah klaim itu benar. Tentukan apakah Composer, npm, dan berkas .env ada di dalam image produksi, lalu simpulkan.

Petunjuk

- Menjalankan perintah di dalam image tidak harus lewat entripoint bawaannya.
- Perintah shell `command -v` memberi tahu apakah sebuah program tersedia.

- Ada perintah Docker yang memperlihatkan riwayat pembentukan image lapis demi lapis.

Konsep yang Dilatih

Multi-stage build · Permukaan serangan image · Verifikasi klaim

Ruang Jawaban

L1.3.1 Perintah apa yang Anda pakai untuk memeriksanya?

L1.3.2 Apakah Composer, npm, dan .env ditemukan di dalam image?

L1.3.3 Berapa selisih ukuran antara stage vendor dan image produksi?

L1.3.4 Kenapa memeriksa dengan ``docker images`` saja tidak cukup untuk membuktikan klaim ini?

L1.4 Menemukan Berkas yang Seharusnya Tidak Ikut

Soal

Sebuah proyek dibangun dengan `.dockerignore` yang hanya berisi satu baris: `.git`.

Identifikasi berkas atau direktori apa saja yang seharusnya TIDAK ikut ke build context, dan jelaskan akibat konkret bila masing-masing tetap ikut.

Petunjuk

- Perhatikan baris "transferring context" di awal keluaran build.
- Pikirkan tiga kategori: rahasia, dependensi yang dibangun ulang, dan artefak runtime.
- Satu berkas yang sering terlupa dibuat oleh Vite saat mode pengembangan.

Konsep yang Dilatih

Build context · `.dockerignore` · Kebocoran rahasia lewat image

Ruang Jawaban

L1.4.1 Sebutkan minimal lima entri yang perlu ditambahkan ke `.dockerignore`.

L1.4.2 Untuk berkas `.env`: apa akibat konkret bila ia ikut masuk ke image?

L1.4.3 Untuk public/hot: apa akibat konkret bila ia ikut masuk ke image produksi?

L1.4.4 Berapa ukuran build context sebelum dan sesudah perbaikan Anda?

Refleksi Hari 1

Pertanyaan berikut tidak diperiksa benar-salah. Tujuannya membantu Anda mengendapkan apa yang baru dipelajari sebelum melanjutkan ke hari berikutnya.

R1.1 Sebutkan satu hal tentang image atau layer yang berbeda dari dugaan Anda sebelum hari ini.

R1.2 Bila rekan Anda bertanya "kenapa harus multi-stage, kan repot?", bagaimana Anda menjawabnya dalam dua kalimat?

R1.3 Berkas apa di proyek Anda sendiri yang selama ini mungkin ikut terbawa ke image tanpa disadari?

Hari 2 — Stack Lengkap dengan Docker Compose

LATIHAN HARI 2

- L2.1 — Menambah Service Baru ke Stack
- L2.2 — Membedakan Container "Up" dan Database "Siap"
- L2.3 — Menentukan Kebijakan Eviction
- L2.4 — Melacak Job yang Gagal
- L2.5 — Cache Konfigurasi di Waktu yang Salah
- L2.6 — Menguji Paritas Dua Web Server
- L2.7 — Mengukur Dampak Penempatan vendor/

L2.1 Menambah Service Baru ke Stack

Soal

Tim ingin menambahkan Mailpit — server SMTP palsu untuk menangkap email saat pengembangan — ke stack Compose yang sudah ada.

Tambahkan service tersebut, hubungkan dengan aplikasi, dan buktikan email yang dikirim Laravel benar-benar tertangkap.

Petunjuk

- Image [axllent/mailpit](#) mendengarkan SMTP di port 1025 dan antarmuka web di port 8025.
- Aplikasi memanggil service lain memakai NAMA service, bukan alamat IP.
- Laravel perlu diberi tahu mailer, host, dan port yang dipakai.

Konsep yang Dilatih

Service Compose · Resolusi nama antar service · Konfigurasi lewat environment

Ruang Jawaban

L2.1.1 Tulis blok YAML service yang Anda tambahkan.

L2.1.2 Variabel environment apa saja yang perlu diubah pada aplikasi?

L2.1.3 Bagaimana Anda membuktikan email benar-benar tertangkap?

L2.1.4 Kenapa MAIL_HOST tidak boleh diisi `localhost`?

L2.2 Membedakan Container "Up" dan Database "Siap"

Soal

Hapus seluruh stack beserta volumenya, lalu jalankan ulang sambil mengamati dengan cermat kapan container database berstatus Up dan kapan ia benar-benar siap menerima koneksi dari aplikasi.

Ukur selisih waktunya dan jelaskan mekanisme apa yang mencegah aplikasi menyambung terlalu dini.

Petunjuk

- Bandingkan kolom STATUS pada `docker compose ps` dengan isi log database.
- Perhatikan tiga baris penanda inisialisasi di log MySQL.
- Perhatikan juga apa yang dikerjakan entrypoint aplikasi sebelum menjalankan php-fpm.

Konsep yang Dilatih

Healthcheck · depends_on: service_healthy · Inisialisasi MySQL

Ruang Jawaban

L2.2.1 Berapa selisih waktu antara container berstatus Up dan berstatus healthy?

L2.2.2 Tiga baris apa di log MySQL yang menandai inisialisasi berjalan lengkap?

L2.2.3 Dua mekanisme apa yang mencegah aplikasi menyambung sebelum database siap?

L2.2.4 Apa yang terjadi bila proses inisialisasi terputus di tengah?

L2.3 Menentukan Kebijakan Eviction

Soal

Untuk masing-masing kasus di bawah, tentukan kebijakan `maxmemory-policy` yang tepat dan jelaskan akibatnya bila salah pilih.

Kasus A: menyimpan hasil query yang mahal, boleh dihitung ulang kapan saja.

Kasus B: menampung job pengiriman invoice yang belum diproses.

Kasus C: menyimpan sesi login pengguna.

Kasus D: satu instance Redis dipakai untuk ketiganya sekaligus.

Petunjuk

- Tanyakan pada tiap kasus: bila data ini hilang, apakah bisa dibangun ulang?
- Ingat bahwa kebijakan eviction berlaku per-server, bukan per-database.

Konsep yang Dilatih

Kebijakan eviction Redis · Pemisahan beban kerja · Ketahanan data

Ruang Jawaban

L2.3.1 Kasus A – kebijakan apa, dan alasannya?

L2.3.2 Kasus B – kebijakan apa, dan apa akibatnya bila salah?

L2.3.3 Kasus C – kebijakan apa, dan apa yang dialami pengguna bila salah?

L2.3.4 Kasus D – apa saran Anda, dan kenapa memisahkan nomor database tidak menyelesaikannya?

L2.4 Melacak Job yang Gagal

Soal

Buat sebuah job yang sengaja selalu gagal, kirim ke antrean, lalu lacak kegagalannya sampai ke pesan kesalahan aslinya.

Jelaskan berapa kali job dicoba ulang dan di mana catatan kegagalannya disimpan.

```
<?php

namespace App\Jobs;

use Illuminate\Contracts\Queue\ShouldQueue;
use Illuminate\Foundation\Queue\Queueable;

class JobGagal implements ShouldQueue
{
    use Queueable;

    public function handle(): void
    {
        throw new \RuntimeException('Sengaja gagal untuk latihan.');
```

Petunjuk

- Job yang melempar exception akan dianggap gagal.
- Horizon menyimpan riwayat job gagal, dan Laravel punya perintah untuk melihatnya.
- Perhatikan opsi `tries` pada konfigurasi Horizon.

Konsep yang Dilatih

Siklus hidup job · Percobaan ulang · Diagnosis lewat log dan dashboard

Ruang Jawaban

L2.4.1 Berapa kali job dicoba sebelum dinyatakan gagal permanen?

L2.4.2 Di mana catatan job gagal disimpan, dan bagaimana cara melihatnya?

L2.4.3 Salin baris pesan kesalahan aslinya.

L2.4.4 Bagaimana cara menjalankan ulang job yang sudah gagal?

L2.5 Cache Konfigurasi di Waktu yang Salah

Soal

Seorang rekan memindahkan `php artisan config:cache` dari entrypoint ke dalam Dockerfile dengan alasan "biar start-nya lebih cepat".

Setelah image di-deploy ke server, aplikasi menyambung ke database yang salah — padahal berkas `.env` di server sudah benar.

Jelaskan penyebabnya, dan rancang cara membuktikannya.

Petunjuk

- Cari tahu apa persisnya yang disimpan `config:cache` ke dalam berkas.
- Pikirkan environment milik siapa yang berlaku saat perintah itu dijalankan.
- Ada perintah artisan yang memperlihatkan nilai konfigurasi yang sedang berlaku.

Konsep yang Dilatih

Cache konfigurasi Laravel · Build time vs runtime · Sumber nilai environment

Ruang Jawaban

L2.5.1 Apa yang sebenarnya disimpan `config:cache` ke dalam berkas?

L2.5.2 Kenapa nilai di server tidak terpakai?

L2.5.3 Rancang langkah untuk membuktikan dugaan Anda.

L2.5.4 Kapan ``config:cache`` seharusnya dijalankan, dan kenapa?

L2.6 Menguji Paritas Dua Web Server

Soal

Stack ini menyediakan dua web server yang dapat dipilih. Susun daftar periksa perilaku, lalu uji KEDUANYA dan buktikan apakah benar-benar setara.

Laporkan setiap perbedaan yang Anda temukan, sekecil apa pun.

Petunjuk

- Uji minimal: halaman utama, endpoint kesehatan, aset statis, berkas unggahan, berkas titik, dan akses langsung ke `index.php`.
- Periksa juga header respons, bukan hanya kode status.
- Berganti profil memerlukan pembersihan lebih dulu.

Konsep yang Dilatih

Paritas konfigurasi · Pengujian perilaku · Header keamanan

Ruang Jawaban

L2.6.1 Tulis daftar periksa yang Anda susun.

L2.6.2 Adakah perbedaan hasil antara kedua web server? Sebutkan.

L2.6.3 Header keamanan apa saja yang muncul pada respons halaman utama?

L2.6.4 Kenapa menguji berkas titik yang TIDAK ADA bukan pengujian yang sah?

L2.7 Mengukur Dampak Penempatan vendor/

Soal

Ukur waktu respons aplikasi dalam mode pengembangan dengan `vendor/` berada di bind mount, lalu ukur ulang setelah `vendor/` dipindah ke named volume.

Laporkan angkanya dan jelaskan penyebab perbedaannya.

Petunjuk

- Ukur setelah aplikasi hangat, bukan pada request pertama.
- Beri jeda beberapa detik antar pengukuran agar OPcache melakukan validasi ulang.
- Perhatikan pengaturan `opcache.revalidate_freq`.

Konsep yang Dilatih

Kinerja bind mount · OPcache validate_timestamps · Named volume

Ruang Jawaban

L2.7.1 Berapa waktu respons dengan vendor/ di bind mount?

L2.7.2 Berapa waktu respons dengan vendor/ di named volume?

L2.7.3 Apa penyebab teknis perbedaan itu?

L2.7.4 Apa konsekuensi memindahkan vendor/ ke named volume, dan bagaimana menangannya?

Refleksi Hari 2

Pertanyaan berikut tidak diperiksa benar-salah. Tujuannya membantu Anda mengendapkan apa yang baru dipelajari sebelum melanjutkan ke hari berikutnya.

R2.1 Apa perbedaan paling penting antara container "Up" dan layanan "siap"?

R2.2 Di proyek Anda sendiri, data apa yang TIDAK boleh dibuang saat memori penuh?

R2.3 Satu hal apa dari hari ini yang akan langsung Anda terapkan minggu depan?

Hari 3 — Kubernetes: Konsep Inti

LATIHAN HARI 3

- L3.1 — Menelusuri Deployment sampai Endpoint
- L3.2 — Perubahan Konfigurasi yang Tidak Berlaku
- L3.3 — Menguji Ketahanan Data
- L3.4 — Memilih Probe yang Tepat
- L3.5 — Membuat Overlay Staging

L3.1 Menelusuri Deployment sampai Endpoint

Soal

Mulai dari satu Deployment, telusuri seluruh rantai objek sampai ke alamat IP pod yang benar-benar menerima trafik.

Lalu hapus satu pod dan jelaskan apa yang terjadi pada rantai itu.

Petunjuk

- Rantainya: Deployment membuat ReplicaSet, ReplicaSet membuat Pod, Service memilih Pod lewat label.
- Objek EndpointSlice memperlihatkan pod mana yang benar-benar terdaftar.
- Perhatikan apakah alamat IP pod pengganti sama dengan yang dihapus.

Konsep yang Dilatih

Deployment · ReplicaSet · Service · EndpointSlice · Pemilihan lewat label

Ruang Jawaban

L3.1.1 Tulis rantai objeknya lengkap dengan nama masing-masing.

L3.1.2 Alamat IP apa saja yang terdaftar di EndpointSlice sebelum penghapusan?

L3.1.3 Setelah satu pod dihapus, apa yang berubah pada nama pod dan alamat IP-nya?

L3.1.4 Kenapa web server tetap bisa menghubungi aplikasi walau alamat IP berubah?

L3.2 Perubahan Konfigurasi yang Tidak Berlaku

Soal

Ubah satu nilai konfigurasi non-rahasia, terapkan, lalu buktikan apakah pod benar-benar memakai nilai yang baru.

Lakukan dua kali: sekali memakai ConfigMap biasa, dan sekali memakai configMapGenerator. Jelaskan perbedaannya.

Petunjuk

- Bandingkan nama ConfigMap sebelum dan sesudah perubahan pada kedua pendekatan.
- Periksa nilai yang benar-benar diterima proses di dalam container, bukan nilai di cluster.

Konsep yang Dilatih

ConfigMap · configMapGenerator · Pemicu rollout · Immutability lewat hash

Ruang Jawaban

L3.2.1 Dengan ConfigMap biasa: apakah nilai di dalam pod ikut berubah? Kenapa?

L3.2.2 Dengan configMapGenerator: apa yang berubah pada nama ConfigMap?

L3.2.3 Kenapa perubahan nama memicu rollout?

L3.2.4 Kenapa pendekatan pertama berbahaya di produksi?

L3.3 Menguji Ketahanan Data

Soal

Buktikan secara mandiri mana data yang bertahan dan mana yang hilang ketika pod dihapus: data MySQL, data Redis queue, data Redis cache, dan berkas unggahan.

Susun tabel hasil pengujian Anda.

Petunjuk

- Tulis penanda yang mudah dikenali ke masing-masing penyimpanan lebih dulu.
- Perhatikan mana yang memakai PVC dan mana yang memakai emptyDir.
- Tunggu pod pengganti benar-benar Ready sebelum memeriksa.

Konsep yang Dilatih

PersistentVolumeClaim · emptyDir · StatefulSet · Ketahanan data

Ruang Jawaban

L3.3.1 Tulis tabel: penyimpanan, jenis volume, bertahan atau hilang.

L3.3.2 Kenapa data Redis cache berperilaku berbeda dari Redis queue?

L3.3.3 Apa yang terjadi pada berkas unggahan, dan kenapa?

L3.3.4 Bagaimana Anda mengetahui jenis volume yang dipakai tiap workload?

L3.4 Memilih Probe yang Tepat

Soal

Untuk setiap skenario di bawah, tentukan probe mana yang tepat dan sebutkan setelan pentingnya. Jelaskan juga apa yang terjadi bila salah pilih.

Skenario A: aplikasi butuh 90 detik untuk booting, sesudah itu responsnya cepat.

Skenario B: aplikasi kadang sibuk sampai 30 detik pada jam sibuk, tetapi tetap sehat.

Skenario C: aplikasi menggantung selamanya setelah kehilangan koneksi ke broker pesan.

Petunjuk

- Tiga probe menjawab tiga pertanyaan berbeda.
- Perhatikan akibat kegagalan masing-masing: dikeluarkan dari Service, atau dibunuh.
- Pikirkan juga apa yang TIDAK boleh disentuh oleh livenessProbe.

Konsep yang Dilatih

startupProbe · readinessProbe · livenessProbe · Kegagalan berantai

Ruang Jawaban

L3.4.1 Skenario A – probe apa, setelan apa, dan apa akibatnya bila salah?

L3.4.2 Skenario B – probe apa, dan kenapa BUKAN livenessProbe yang diperketat?

L3.4.3 Skenario C – probe apa, dan kenapa probe lain tidak menyelesaikannya?

L3.4.4 Kenapa livenessProbe sebaiknya tidak menyentuh database?

L3.5 Membuat Overlay Staging

Soal

Buat overlay baru bernama `staging` yang berbeda dari overlay lokal dalam tiga hal: namespace `laravel-staging`, jumlah replika aplikasi 1, dan LOG_LEVEL `debug`.

Base TIDAK boleh diubah sama sekali.

Petunjuk

- Overlay merujuk base lewat [resources](#).
- Untuk mengubah satu field pada objek tertentu, gunakan patch.
- Periksa hasilnya tanpa menerapkannya lebih dulu.

Konsep yang Dilatih

Kustomize overlay · Patch · Pemisahan lingkungan

Ruang Jawaban

L3.5.1 Tulis isi `kustomization.yaml` overlay staging Anda.

L3.5.2 Bagaimana Anda mengubah `LOG_LEVEL` tanpa menyentuh base?

L3.5.3 Perintah apa yang Anda pakai untuk memeriksa hasilnya sebelum menerapkan?

L3.5.4 Kenapa base sebaiknya tidak pernah diubah untuk kebutuhan satu lingkungan?

Refleksi Hari 3

Pertanyaan berikut tidak diperiksa benar-salah. Tujuannya membantu Anda mengendapkan apa yang baru dipelajari sebelum melanjutkan ke hari berikutnya.

R3.1 Jelaskan dengan kalimat sendiri hubungan Deployment, ReplicaSet, Pod, dan Service.

R3.2 Kenapa mengubah ConfigMap saja tidak cukup untuk mengubah perilaku aplikasi?

R3.3 Probe mana yang menurut Anda paling sering disalahgunakan, dan kenapa?

Hari 4 — Produksi dan Keamanan

LATIHAN HARI 4

- L4.1 — Memperbaiki Manifest yang Ditolak
- L4.2 — Menemukan Direktori yang Perlu Writable
- L4.3 — Menguji Apakah NetworkPolicy Benar-benar Ditegakkan
- L4.4 — Mengukur Kegagalan Request Selama Rollout
- L4.5 — Analisis Akar Masalah dari Log

L4.1 Memperbaiki Manifest yang Ditolak

Soal

Manifest berikut ditolak oleh namespace berlabel Pod Security restricted.

Baca pesan penolakannya, lalu perbaiki manifest sampai diterima — tanpa menurunkan tingkat keamanan namespace.

```
apiVersion: v1
kind: Pod
metadata:
  name: latihan-pss
  namespace: laravel
spec:
  containers:
    - name: alat
      image: busybox
      command: ["sh", "-c", "sleep 300"]
```

Petunjuk

- Pesan penolakan menyebut setiap pelanggaran satu per satu.
- Sebagian setelan berada di tingkat pod, sebagian di tingkat container.
- Image busybox berjalan sebagai root secara bawaan.

Konsep yang Dilatih

Pod Security Admission · securityContext · Membaca pesan penolakan

Ruang Jawaban

L4.1.1 Salin pesan penolakan yang Anda terima.

L4.1.2 Ada berapa pelanggaran, dan apa saja?

L4.1.3 Tulis manifest yang sudah diperbaiki.

L4.1.4 Kenapa menurunkan label namespace menjadi `baseline` bukan penyelesaian yang benar?

L4.2 Menemukan Direktori yang Perlu Writable

Soal

Sebuah container dijalankan dengan `readOnlyRootFilesystem: true` dan gagal.

Tentukan direktori mana saja yang benar-benar perlu ditulis, buktikan satu per satu, lalu pasang hanya direktori itu sebagai volume — bukan mematikan setelan keamanannya.

Petunjuk

- Pesan kesalahan biasanya menyebut path yang gagal ditulis.
- Laravel menulis ke beberapa tempat: cache framework, log, dan berkas sementara.
- Beberapa perkakas menulis ke direktori home pengguna.

Konsep yang Dilatih

readOnlyRootFilesystem · emptyDir · Hak akses minimal

Ruang Jawaban

L4.2.1 Direktori apa saja yang perlu writable? Sebutkan beserta alasannya.

L4.2.2 Bagaimana Anda membuktikan sebuah direktori benar-benar tidak bisa ditulis?

L4.2.3 Perkakas apa yang gagal karena direktori home tidak writable, dan bagaimana solusinya?

L4.2.4 Kenapa mematikan readOnlyRootFilesystem bukan penyelesaian yang benar?

L4.3 Menguji Apakah NetworkPolicy Benar-benar Ditegakkan

Soal

Seluruh NetworkPolicy sudah diterapkan tanpa error. Jangan percaya begitu saja.

Rancang dan jalankan pengujian yang membuktikan apakah kebijakan itu benar-benar memblokir, lalu simpulkan.

Petunjuk

- Pod yang tidak berlabel apa pun seharusnya tidak diizinkan menghubungi database.
- Pod uji tetap harus memenuhi Pod Security restricted agar bisa dibuat.
- Kemampuan menegakkan NetworkPolicy datang dari komponen jaringan cluster, bukan dari manifest-nya.

Konsep yang Dilatih

NetworkPolicy · CNI · Verifikasi klaim keamanan

Ruang Jawaban

L4.3.1 Bagaimana rancangan pengujian Anda?

L4.3.2 Apa hasilnya di Docker Desktop?

L4.3.3 Apa kesimpulan Anda tentang postur keamanan cluster ini?

L4.3.4 Kenapa manifest yang ter-apply tanpa error tidak membuktikan apa pun?

L4.4 Mengukur Kegagalan Request Selama Rollout

Soal

Ukur secara kuantitatif berapa request yang gagal selama rolling update.

Lalu ubah satu setelan agar rollout MENJADI menyebabkan kegagalan, ukur ulang, dan jelaskan setelan mana yang paling menentukan.

Petunjuk

- Kirim request terus-menerus dari satu jendela sementara rollout berjalan di jendela lain.
- Perhatikan `maxUnavailable`, `readinessProbe`, dan jeda `preStop`.

- Untuk membuat rollout gagal, cukup ubah satu di antaranya.

Konsep yang Dilatih

Rolling update · maxUnavailable · preStop · Pengukuran keandalan

Ruang Jawaban

L4.4.1 Berapa request yang gagal pada konfigurasi bawaan? Dari berapa total?

L4.4.2 Setelan apa yang Anda ubah untuk membuat rollout menyebabkan kegagalan?

L4.4.3 Berapa request yang gagal setelah perubahan itu?

L4.4.4 Setelan mana yang paling menentukan, dan kenapa?

L4.5 Analisis Akar Masalah dari Log

Soal

Sebuah stack Kubernetes menunjukkan gejala berikut. Database berstatus Running dan sehat, tetapi aplikasi ditolak dengan pesan:

SQLSTATE[HY000] [1130] Host '10.1.0.187' is not allowed to connect to this MySQL server

Tim sudah mencoba menghapus PVC dan menerapkan ulang manifest, tetapi masalahnya kembali.

Temukan akar masalahnya dan usulkan perbaikan permanen.

Petunjuk

- Log container yang SEKARANG berjalan belum tentu memuat kejadian awalnya.
- Perhatikan apakah log memuat penanda inisialisasi database.
- Periksa variabel environment yang disuntikkan ke container database.
- Cari tahu apakah ada program lain yang membaca variabel itu tanpa diminta.

Konsep yang Dilatih

Diagnosis akar masalah · Log container sebelumnya · Efek samping environment

Ruang Jawaban

L4.5.1 Perintah apa saja yang Anda jalankan, berurutan?

L4.5.2 Apa yang Anda temukan di log container SEBELUM restart terakhir?

L4.5.3 Apa akar masalahnya?

L4.5.4 Kenapa menghapus PVC tidak menyelesaikannya, dan apa perbaikan permanennya?

Refleksi Hari 4

Pertanyaan berikut tidak diperiksa benar-salah. Tujuannya membantu Anda mengendapkan apa yang baru dipelajari sebelum melanjutkan ke hari berikutnya.

R4.1 Klaim keamanan apa yang selama ini Anda percayai tanpa pernah mengujinya?

R4.2 Dari empat studi kasus kegagalan, mana yang paling mungkin terjadi di lingkungan Anda?

R4.3 Satu perubahan apa yang akan Anda usulkan ke tim setelah pelatihan ini?